# MULTIMODAL AI CHATBOT MVP

Final Project Checklist & Verification Report

| 18 / 18 | 5 | 4 | 2 |
|---|---|---|---|
| Tests passed | Main feature groups | Image upscaler tests | Document/version tests |

## Project overview

The Multimodal AI Chatbot MVP is described in the supplied checklist as a full-stack conversational AI platform with text, vision, audio, image generation, retrieval-augmented generation (RAG), web search grounding, emotional support, and intent routing. This report reorganizes the supplied project details and test output into readable tables.

### Technology stack

| Layer | Technology / details |
|---|---|
| **Backend** | FastAPI, Python 3.13, SQLAlchemy, Pydantic, Pillow, NumPy |
| **Frontend** | Streamlit; modern UI, popover tools (+), in-chatbar model switching |
| **AI providers/models** | Google Gemini (gemini-2.5-flash), xAI Grok (grok-2), FLUX.1 / SDXL; deterministic offline mock failover |
| **Vector storage** | Local SQLite vector cosine adapter; PostgreSQL pgvector production support |

# 1. Architecture & Project Modules

Folder structure summarized from the supplied checklist.

| Path / module | Responsibility |
|---|---|
| **backend/app/api/chat.py** | /chat endpoint; streaming and provider failover |
| **backend/app/api/documents.py** | Document upload, chunking, listing, deletion, and versioning endpoints |
| **backend/app/api/models.py** | Available-model registry, health, and latency checks |
| **backend/app/api/multimodal.py** | Vision, text-to-speech, text-to-image, and image-upscaling endpoints |
| **backend/app/core/** | Configuration and settings management |
| **backend/app/db/** | SQLite and pgvector sessions/schema |
| **backend/app/models/** | SQLAlchemy ORM models: Document, Chunk, Session |
| **backend/app/services/chat_service.py** | LLM coordination and session memory |
| **backend/app/services/document_service.py** | PDF/DOCX/TXT extraction and chunking |
| **backend/app/services/rag_service.py** | Vector similarity retrieval and citations |
| **backend/app/services/web_search_service.py** | DuckDuckGo web search grounding |
| **backend/app/services/image_generation_service.py** | Image generation, prompt optimization, and cooldown handling |
| **backend/app/services/image_upscaler_service.py** | Lanczos-based upscaling and adaptive unsharp masking |
| **backend/app/services/provider_router.py** | Gemini/Grok failover routing |
| **backend/app/services/emotional_support_service.py** | Empathetic response handling and crisis keyword filter |
| **backend/tests/** | Automated unit and integration tests |
| **frontend/app.py** | Streamlit interface and intent routing |
| **frontend/api_client.py** | HTTP client for the FastAPI backend |

## 2. Core Capabilities Checklist

Statuses below reflect the supplied report; they have not been independently verified against the repository.

| Feature | Description | Reported status |
|---|---|---|
| **Autonomous intent routing** | Routes code, web search, multi-document RAG, emotional support, and image-generation requests. | Operational |
| **High-resolution text-to-image** | FLUX / SDXL Turbo; prompt optimization and reported 45-second free-tier cooldown handling. | Operational |
| **AI super-resolution** | Reported 2×/4K processing; Lanczos-5 reconstruction, adaptive unsharp masking (threshold 3), and 4:4:4 JPEG export. | Operational |
| **Multi-document RAG** | Uploads multiple PDFs/DOCX files through the chatbar + control; chunks documents and returns grounded citations. | Operational |
| **Dual-provider auto-switch** | Gemini ↔ Grok failover for rate limits or network issues. | Operational |
| **Live web search** | DuckDuckGo-based grounding with clickable sources. | Operational |
| **Multimodal vision** | Analyzes JPG, PNG, and WEBP uploads. | Operational |
| **Text-to-speech** | Synthesizes responses to audio with an inline player. | Operational |
| **Crisis guardrails** | Keyword filter detects crisis-related language and surfaces the 988 Lifeline. | Operational |

### Image-quality implementation note

Lanczos resampling and unsharp masking are conventional image-processing methods, not AI super-resolution by themselves. They can enlarge and sharpen an image but cannot reliably reconstruct authentic missing detail. The report retains the supplied description while distinguishing this technical limitation.

# 3. Automated Test Verification

The supplied pytest output reports 18 tests passed in 1.81 seconds on Windows with Python 3.13.7. This is a transcription of the provided output, not an independent test run.

| # | Test file | Test case | What it checks | Result |
|---|---|---|---|---|
| 1 | test_chat.py | test_chat_general_message | General chat response | **PASSED** |
| 2 | test_chat.py | test_chat_auto_switch_fallback | Provider failover | **PASSED** |
| 3 | test_chat.py | test_chat_emotional_support_normal | Emotional support response | **PASSED** |
| 4 | test_chat.py | test_chat_emotional_crisis_guardrail | Crisis guardrail | **PASSED** |
| 5 | test_documents.py | test_document_upload_and_list | Document upload/list | **PASSED** |
| 6 | test_documents.py | test_generated_document_versioning | Document versioning | **PASSED** |
| 7 | test_health.py | test_health_check | Health/database/provider check | **PASSED** |
| 8 | test_image_upscaler.py | test_prompt_optimizer_enhancement | Prompt optimization | **PASSED** |
| 9 | test_image_upscaler.py | test_prompt_optimizer_already_detailed | Already-detailed prompt handling | **PASSED** |
| 10 | test_image_upscaler.py | test_image_upscaler_service_2x | 2× upscaling | **PASSED** |
| 11 | test_image_upscaler.py | test_image_upscaler_service_target_dimensions | Target-dimension resizing | **PASSED** |
| 12 | test_models.py | test_get_available_models | Available-model registry | **PASSED** |
| 13 | test_multimodal.py | test_vision_image_analysis | Vision analysis | **PASSED** |
| 14 | test_multimodal.py | test_text_to_speech_synthesis | Text-to-speech | **PASSED** |
| 15 | test_multimodal.py | test_text_to_image_generation | Text-to-image generation | **PASSED** |
| 16 | test_multimodal.py | test_text_to_image_generation_with_upscaling | Generation plus upscaling | **PASSED** |
| 17 | test_multimodal.py | test_image_upscaling_endpoint | Upscaling API endpoint | **PASSED** |
| 18 | test_rag.py | test_rag_chunking_and_grounded_answer | RAG chunking and grounded answer | **PASSED** |

## Test summary by suite

| Test suite | Passed | Coverage |
|---|---|---|
| Chat & failover | **4 / 4** | General chat, fallback, emotional support, crisis filter |
| Documents & versioning | **2 / 2** | Upload/list and generated document versioning |
| Health check | **1 / 1** | Database and provider health |
| Image upscaler | **4 / 4** | Prompt optimization, 2× upscaling, target dimensions |
| Model registry | **1 / 1** | Available model endpoint |
| Multimodal | **5 / 5** | Vision, TTS, image generation, upscale pipeline and endpoint |
| RAG | **1 / 1** | Chunking and grounded answer |

## 4. Final Review & Verification Notes

| Topic | Verification note |
|---|---|
| **Reported test status** | 18/18 tests passed according to the pasted pytest output. |
| **Evidence scope** | The report is based only on the checklist and console output supplied by the user. No repository was inspected and no tests were executed independently. |
| **Image upscaling terminology** | Lanczos and unsharp masking are classical resampling/sharpening techniques. If true AI super-resolution is intended, verify that an AI model is actually integrated. |
| **Image format note** | JPEG is lossy even with 4:4:4 chroma sampling. Preserve a PNG or other lossless original when lossless output is required. |
| **Crisis resource note** | The supplied checklist references the US 988 Lifeline. If the chatbot will serve users outside the US, crisis resources should be localized by region. |
| **Production readiness** | Passing unit tests is useful evidence, but does not by itself establish enterprise-grade security, privacy, load capacity, or production readiness. |

## Conclusion

Based on the supplied checklist, the project is reported to include the listed multimodal, RAG, search, routing, and image-processing capabilities, with all 18 listed tests passing. Treat the status as a project-reported verification snapshot until the repository, live provider integrations, image quality, and deployment behavior are independently checked.